DECK 03 · WEEK 3

Tokens & Token-2022

This week you build tokens in practice: standard SPL first, then Token-2022 with extensions.

TOKENS ARE THE CORE BUILDING BLOCK

DeFi, NFTs, governance and payments all depend on token accounts and token programs.

Exercise 3 · Your First Token

Create an SPL mint, create token accounts, mint supply, transfer between wallets, then script balance reads in TypeScript.

`OUTPUT: WORKING SPL TOKEN FLOW`

Exercise 4 · Token-2022 Extensions

Create a Token-2022 mint with extensions, test extension behavior, and decode extension data from the mint account.

`OUTPUT: EXTENDED TOKEN MINT`

AI SUPPORT STEPS DOWN

Guidance is lighter than Week 2. You get prompts and pitfalls, but you're expected to decide more of the implementation details.

THREE TOKEN ACCOUNT TYPES

Your wallet does not ”contain” tokens. Token accounts do.

Mint account

Defines the token: total supply, decimals, and mint authority. One mint account per token.

Token account

Holds one wallet’s balance of one mint. Three different tokens means three token accounts.

Associated Token Account

Deterministic token account derived from wallet + mint. Standard way to locate balances.

COMMON MIX-UP

Fetching a token balance from a wallet address fails. Always derive or look up the ATA first.

CLI-FIRST TOKEN CREATION

Use the `spl-token` CLI to create, mint, and inspect before writing code.

01 Create a mint and save the mint address.

02 Create your wallet ATA for that mint.

03 Mint a round amount such as `1000`.

04 Verify holdings with `spl-token accounts`.

QUICK BALANCE CHECK

```
spl-token accounts
```

Then inspect the mint on Explorer (devnet): supply, decimals, and mint authority should match your intent.

SAVE THE METADATA YOU WILL REUSE

Record public token metadata once so scripts and apps don’t drift between runs.

CONFIG/TOKENS.DEVNET.JSON

```
{
  "cluster": "devnet",
  "tokens": {
    "token1": {
      "mint": "NEW_MINT_HERE",
      "ownerTokenAccount":
"NEW_TOKEN_ACCOUNT_HERE",
      "decimals": 9
    }
  }
}
```

Store these values

`mint`, `ownerTokenAccount`, `decimals`, cluster, and useful transaction signatures for debugging.

!

NEVER STORE SECRETS

Do not commit private keys, seed phrases, or keypair JSON contents in project files.

TWO WALLETS, ONE MINT

Create a second wallet, transfer 100 tokens, and verify both balances.

1 **SECOND WALLET**

Generate a keypair with `solana-keygen new --outfile ~/second-wallet.json` and airdrop SOL for rent fees.

2 **CREATE/USE ATA**

Create or target recipient ATA for your mint, then run a transfer of 100 tokens.

3 **VERIFY END STATE**

Main wallet should drop by 100, recipient should receive 100. Confirm through CLI and Explorer.

↯ **UNDER THE HOOD**

If recipient ATA doesn't exist, CLI may perform ATA creation + transfer as separate operations.

TYPESCRIPT CHECKPOINT

Read token balances.

DERIVE ATA · FETCH ACCOUNT · FORMAT UNITS

SCRIPT REQUIREMENTS

Connect to devnet, derive ATAs, fetch balances, print readable values.

MUST INCLUDE

01 Mint address + two wallet addresses (hardcoded is fine).

02 `getAssociatedTokenAddress` for each wallet.

03 `getAccount` to fetch token account data.

04 Readable output for both balances.

DECIMALS CHECK

Raw token amounts are base units. Divide by `10^decimals` to display user-facing token values.

AI PITFALL

Generated scripts often hardcode 9 decimals or print raw integers. Read decimals from the mint.

WHERE AI HELPS VS WHERE TO VERIFY

AI is strong on CLI syntax and boilerplate, weaker on your local token/account state.

Good AI use

Generate command syntax, TypeScript scaffolding, and diagnostics for missing ATA or wrong address inputs.

Verify manually

Program IDs, package versions, decimals logic, and whether addresses are wallet keys or token accounts.

Frequent failure

`account not found` because ATA was never created, or script tries to read wallet address as token account.

Done when

CLI and script both report consistent balances and Explorer confirms supply + ownership fields.

TOKEN-2022 EXTENSIONS

Programmable token behavior.

EXTENSIONS ARE BAKED IN AT CREATION

Token-2022 is a different program from classic SPL Token, with additional mint/account data and behaviors.

Metadata

Name, symbol, and URI can live directly on the mint, without a separate metadata account.

Transfer fee

Each transfer withholds a configured fee, collected later by the fee authority.

Non-transferable

Tokens can be minted and burned but cannot be transferred between wallets.

Default frozen state

New accounts start frozen and require authority action before transfers are allowed.

!

PERMANENT CHOICE

You cannot add extensions to an existing mint. If config changes, create a new mint.

BUILD IT, THEN CONTRAST WITH EXERCISE 3

The goal is understanding account-level differences between standard SPL and Token-2022 mints.

01 Create a Token-2022 mint with at least two extensions.

02 Mint supply and test behavior (fee withheld, transfer blocked, freeze/thaw, etc.).

03 Inspect with `spl-token display <MINT_ADDRESS>`.

04 Write a TS script to decode extension settings from mint account data.

COMPARE	SPL TOKEN	TOKEN-2022
	`Tokenkeg...`	`TokenzQd...`
	Standard fields only	Standard + extension fields
	Fixed primitive token logic	Programmable extension logic

TOKEN-2022 IS WHERE STALE OUTPUT SHOWS UP

Use AI for parameter-heavy setup, but cross-check current docs for compatibility and flags.

AI HELPS WITH

01 Generating CLI commands or scripts with extension parameters.

02 Decoding extension bytes in TypeScript.

03 Explaining extension use cases before you pick combinations.

YOU MUST VERIFY

01 Program ID targets Token-2022, not classic SPL.

02 Extensions are configured at mint creation time.

03 Chosen extension set is compatible.

04 CLI flags match current documentation.

WHAT YOU LEAVE WITH

You can create and reason about both standard and extended token systems on Solana.

SPL token workflow

Created, minted, transferred, and inspected with CLI + Explorer.

Balance script

TypeScript script derives ATAs, reads accounts, and formats balances using decimals.

Token-2022 practice

Mint with extensions configured and behavior validated.

Account-level comparison

You can explain mint structure, program ownership, and capability differences.